

Computerwoche-Inhalte schnell finden und dank KI die richtigen Antworten erhalten? Testen Sie unsere neue hybride Suchfunktion! **Jetzt ausprobieren!**

COMPUTERWOCHE

Home • Artificial Intelligence



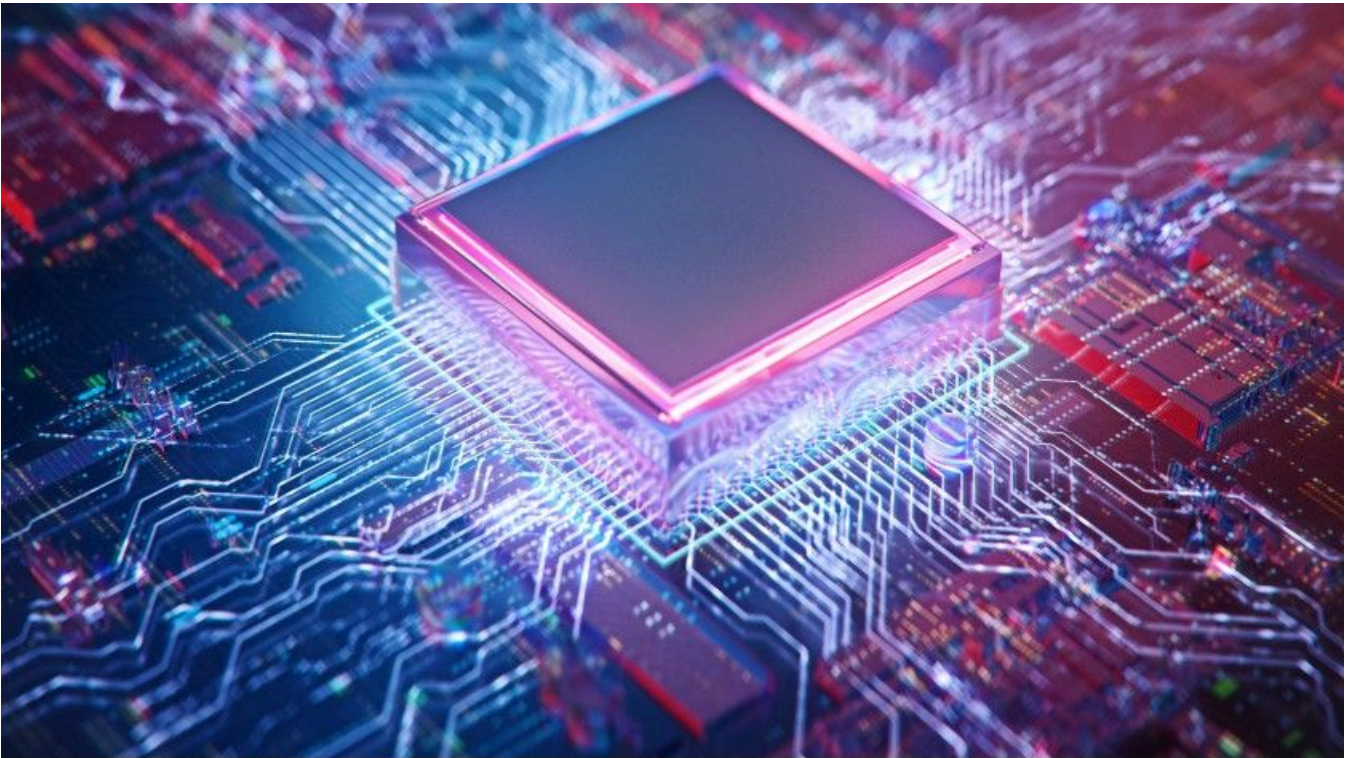
von Ian Pointer

Künstliche Intelligenz programmieren: Die besten Coding-Sprachen für KI

Tipp

26. Juni 2026 • 9 Minuten

Sie wissen nicht, welche Programmiersprache Sie für Ihr KI-Projekt verwenden sollen? Wir zeigen Ihnen die besten Coding-Optionen in Sachen Künstliche Intelligenz.



Wenn es darum geht, Künstliche Intelligenz zu programmieren, stehen Ihnen diverse Optionen zur Wahl. Wir zeigen Ihnen die besten KI-Programmiersprachen.

Foto: Connect world – shutterstock.com

Künstliche Intelligenz [<https://www.computerwoche.de/article/2769837/was-sie-zum-thema-ki-wissen-muessen.html>] (KI) eröffnet Softwareentwicklern völlig neue Möglichkeiten: Mit Hilfe von **Machine und Deep Learning** [<https://www.computerwoche.de/article/2789087/noch-viel-zu-tun-bei-ki-und-ml.html>] lassen sich bessere Nutzerprofile und Empfehlungen, ein höherer Personalisierungsgrad, smartere Suchoptionen oder intelligentere Interfaces realisieren. Dabei stellt sich unweigerlich die Frage, welche Programmiersprache dafür zum Einsatz kommen soll. Die Anforderungen, denen eine KI-Coding-Sprache genügen muss, sind vielfältig: eine Vielzahl von Machine- und Deep-Learning-Bibliotheken sollten genauso vorhanden sein wie eine performante Laufzeitumgebung, ausgiebiger Tool Support, eine große Entwickler-Community und ein gesundes Ökosystem.

Trotzdem dieser Anforderungskatalog umfassend ist, stehen Ihnen einige gute Optionen zur Wahl, wenn es darum geht, Künstliche Intelligenz zu **programmieren** [<https://www.computerwoche.de/article/2813209/11-wege-ihre-softwareentwicklung-neu-zu-definieren.html>]. Wir zeigen Ihnen eine Auswahl der besten KI-Programmiersprachen.

Python

Wenn Sie als Developer mit künstlicher Intelligenz arbeiten, führt mit an **Sicherheit** grenzender Wahrscheinlichkeit kein Weg an **Python** [<https://www.cowo.de/a/3548847>] vorbei. Inzwischen unterstützen auch so gut wie alle gängigen Bibliotheken Python 3.x – die Zeiten, in denen die Umstellung von Python 2.x auf 3.x Kompatibilitätsprobleme mit sich brachte, sind so gut wie vorbei. Mit anderen Worten: Sie können nun endlich auch in der Praxis von den zahlreichen neuen Features von Python 3.x profitieren. Was nicht heißen

soll, dass die packaging-Hürden bei **Python** [<https://www.computerwoche.de/article/2762025/python-lernen-leicht-gemacht.html>] überhaupt keine Rolle mehr spielen – das Gros der Probleme lässt sich aber mit Hilfe von Anaconda umschiffen. Nichtsdestotrotz wäre es zu begrüßen, wenn die Python Community endlich voll und ganz von diesen Hürden befreit würde.

Davon abgesehen sind die verfügbaren mathematischen und statistischen Bibliotheken von Python denen anderer Programmiersprachen weit voraus: **NumPy** [<https://numpy.org/>] ist inzwischen so allgegenwärtig, dass es beinahe als Standard-API für Tensor Operations bezeichnet werden kann, während **Pandas** [<https://pandas.pydata.org/>] die flexiblen Dataframes von R in die Python-Welt trägt. Geht es um Natural Language Processing (NLP) haben Sie die Wahl zwischen dem altherwürdigen **NLTK** [<https://www.nltk.org/>] und dem superschnellen **SpaCy** [<https://spacy.io/>], während sich für Machine-Learning-Zwecke das bewährte **scikit-learn** [<https://scikit-learn.org/stable/>] empfiehlt. Geht es hingegen um Deep Learning, sind alle aktuellen Bibliotheken (**TensorFlow** [<https://www.tensorflow.org/>], **PyTorch** [<https://pytorch.org/>], **Chainer** [<https://chainer.org/>], **Apache MXNet** [<https://mxnet.apache.org/>], etc.) im Grunde "Python-first"-Projekte.

Wenn Sie ein regelmäßiger Besucher von **arXiv** [<https://arxiv.org/>] sind, wird Ihnen längst aufgefallen sein, dass die Mehrzahl der dortigen Deep-Learning-Forschungsprojekte, die Quellcode zur Verfügung stellen, dazu auf Python setzen. In Sachen Deployment-Modelle haben Microservice-Architekturen und -Technologien wie **SeldonCore** [<https://github.com/seldonio/seldon-core>] die Auslieferung von Python-Modellen in Produktivumgebungen wesentlich vereinfacht.

Python [<https://www.computerwoche.de/article/2762145/python-besitzt-mehr-als-300-bibliotheken.html>] ist zweifellos die Programmiersprache der Wahl, wenn es um KI-Forschung geht: Sie bietet die größte Auswahl an Machine und Deep Learning Frameworks und ist die Coding-Sprache, die innerhalb der KI-Welt tonangebend ist.

C++

C++ [<https://www.computerwoche.de/article/2816926/wie-sich-c-gegen-c-python-und-co-schlaegt.html>] ist aller Voraussicht nach nicht die erste Wahl für Ihr **KI-Projekt** [<https://www.computerwoche.de/article/2781630/so-wird-ihr-ki-projekt-ein-erfolg.html>]. Allerdings wird Deep Learning im Edge-Bereich ein immer gängigeres Szenario. In diesem Fall müssen Sie Ihre Modelle auf Systemen zum Laufen bringen, die nur sehr begrenzte Ressourcen zur Verfügung haben. Um das letzte bisschen Performance aus dem System zu pressen, kann es nötig werden, noch einmal in die Untiefen der Pointer-Welt abzutauchen.

Glücklicherweise kann moderner C++ Code aber tatsächlich angenehm zu schreiben sein. Hierfür stehen Ihnen mehrere Ansätze zur Wahl: Entweder Sie nutzen Bibliotheken wie Nvidias **CUDA** [<https://www.computerwoche.de/article/2818437/was-ist-cuda.html>] um ihren eigenen Programmcode zu schreiben, der direkt in die GPU fließt – oder Sie setzen wahlweise auf TensorFlow oder PyTorch, um Zugang zu flexiblen high-level **APIs** [<https://w>

www.computerwoche.de/article/4004872/die-besten-apis-um-ki-zu-integrieren.html] zu erlangen. Sowohl **PyTorch** [<https://www.computerwoche.de/article/2794453/5-gruende-fuer-das-deep-learning-framework.html>] als auch **TensorFlow** [<https://www.computerwoche.de/article/2789330/die-besten-tools-fuer-tensorflow.html>] erlauben Ihnen, Modelle, die in Python geschrieben sind, in eine C++ Laufzeitumgebung zu integrieren. So rücken Sie deutlich näher an den Produktiveinsatz, bleiben dabei aber flexibel in der Entwicklung.

Weil KI-Applikationen sich immer stärker über alle Devices – von Embedded Systems bis hin zu riesigen Clustern – hinweg ausbreiten, ist C++ ein wichtiger Bestandteil des KI-Coding-Toolkits. Um **künstliche Intelligenz im Edge-Bereich** [<https://www.computerwoche.de/article/2807255/so-sichern-sie-den-netzwerkrand-ab.html>] zu realisieren, gilt es eben nicht nur akkurat zu programmieren, sondern auch qualitativ gut und schnell.

Java und andere JVM-Sprachen

Die Familie der JVM-Programmiersprachen (**Java** [<https://www.computerwoche.de/article/2814735/warum-java-immer-noch-rockt.html>], Scala, **Kotlin** [<https://www.computerwoche.de/article/2817523/was-ist-kotlin.html>], Clojure, etc.) ist weiterhin eine gute Wahl, wenn es um die Entwicklung von KI-Applikationen geht. Eine reichhaltige Auswahl an Bibliotheken steht für nahezu alle Aspekte zur Auswahl – sei es Natural Language Processing (**CoreNLP** [<https://stanfordnlp.github.io/CoreNLP/>]), Tensor Operations oder GPU-beschleunigtes Deep Learning (**DL4J** [<https://deeplearning4j.org/>]). Darüber hinaus gewährleisten diese Coding-Sprachen auch einfachen Zugang zu Big-Data-Plattformen wie **Apache Spark** [<https://www.infoworld.com/article/2259224/what-is-apache-spark-the-big-data-platform-that-crushed-hadoop.html>] und **Apache Hadoop** [<https://hadoop.apache.org/>].

Computerwoche Smart Answers [Mehr erfahren](#)

Verwandte Themen erkunden

- [Welche Rolle spielen TensorFlow und PyTorch als Open-Source-KI-Tools?](#)
- [Wie kann ich meine Python-KI-Anwendungen effektiv skalieren?](#)
- [Welche Hardware wird für verschiedene KI-Anwendungsbereiche benötigt?](#)
- [Welche Möglichkeiten bietet JavaScript für die Entwicklung von KI-Anwendungen?](#)
- [Was sind aktuelle Herausforderungen im Python-Ökosystem für KI-Entwickler?](#)

Eine Frage stellen

FRAGEN

Für die meisten Unternehmen ist **Java** [<https://www.computerwoche.de/article/2814678/darum-h-eisst-java-java.html>] die lingua franca – und mit **Java** 8 und neueren Versionen verliert auch die Erstellung von Java Code ihren Schrecken. Eine **KI-Applikation** [<https://www.computerwoche.de/article/2793583/kuenstliche-intelligenz-verdraengt-den-menschen-nicht.html>] in Java zu programmieren mag sich ein wenig langweilig anfühlen, sorgt aber in der Regel für zufriedenstellende Ergebnisse und ermöglicht Ihnen, alle existierenden Bestandteile einer Java-Infrastruktur für Entwicklung, Deployment und Monitoring einzusetzen.

JavaScript

JavaScript [<https://www.computerwoche.de/article/2832952/was-ist-javascript.html>] ausschließlich für die Entwicklung von KI-Applikationen zu erlernen, ist ein höchst unwahrscheinliches Szenario. Allerdings bietet Googles **TensorFlow.js** [<https://www.tensorflow.org/js>] weiterhin eine gute Möglichkeit, Ihre Keras- und TensorFlow-Modelle über Browser oder Node.js auszuliefern.

Dennoch ist der große Ansturm von **JavaScript-Entwicklern** [<https://www.computerwoche.de/article/2794625/was-javascript-von-typescript-unterscheidet.html>] im Bereich **Künstliche Intelligenz** [<https://www.computerwoche.de/k/kuenstliche-intelligenz-artificial-intelligence,3544>] bislang ausgeblieben. Das könnte daran liegen, dass das JavaScript-Ökosystem in Sachen verfügbare Bibliotheken bislang den nötigen Tiefgang vermissen lässt – zumindest im Vergleich zu Programmiersprachen wie Python. Darüber hinaus stehen auf Serverseite durch Deployment-Modelle mit Node.js (wiederum im Vergleich zu den Python-Optionen) keine wirklichen Vorteile in Aussicht. KI-Applikationen auf JavaScript-Basis dürften deshalb auch weiterhin auf Browser-Basis entstehen.

Swift

Swift for TensorFlow [<https://www.tensorflow.org/swift>] verbindet die neuesten und besten Features von TensorFlow mit den Vorteilen von Python-Bibliotheken, die sich problemlos einbinden lassen – ganz so als würden Sie Python selbst nutzen.

Das Team von fast.ai werkelt derzeit an einer Swift-Version seiner populären Bibliothek – und stellt zahlreiche Optimierungen in Aussicht, gerade in Zusammenhang mit dem **LLVM compiler** [<https://www.computerwoche.de/article/2826586/was-ist-llvm.html>]. Von “production ready” kann zwar noch keine Rede sein, aber auf dieser Grundlage könnte

die nächste Generation von Deep-Learning-Entwicklungsarbeit entstehen – Sie sollten Swift deshalb auf alle Fälle im Auge behalten.

R

R [<https://www.r-project.org/>] ist die Programmiersprache der Wahl für **Data Scientists** [<https://www.computerwoche.de/article/2774747/was-data-scientists-koennen-muessen.html>]. Developer aus anderen Bereichen könnten die Coding-Sprache wegen ihres Dataframe-zentrischen Ansatzes hingegen als verwirrend empfinden.

Für ein Team leidenschaftlicher R-Entwickler kann es durchaus Sinn machen, Integrationen mit **TensorFlow** [<https://www.tensorflow.org/>], **Keras** [<https://keras.io/>] oder **H2O** [<https://www.h2o.ai/>] für Forschung und Prototyping einzusetzen. Hinsichtlich der Performance ist R für den Produktiveinsatz aber lediglich bedingt zu empfehlen. Zwar lässt sich performanter R Code durchaus produktiv zum Einsatz bringen, einfacher dürfte es aber in den allermeisten Fällen sein, den R-Prototypen in **Java** oder Python neu zu programmieren.

KI programmieren – weitere Optionen

Natürlich sind die vier genannten Programmiersprachen nicht die einzigen Optionen, um **Künstliche Intelligenz** [<https://www.computerwoche.de/k/kuenstliche-intelligenz-artificial-intelligence,3544>] zu programmieren. Die folgenden beiden Coding-Sprachen könnten – je nach Einsatzzweck – ebenfalls von Interesse für Ihre **KI-Projekte** [<https://www.computerwoche.de/article/2792741/ki-bewaehrt-sich-in-der-praxis.html>] sein:

Lua [<https://www.lua.org/>]

Vor einigen Jahren wurde Lua als “next big thing” im Bereich der Künstlichen Intelligenz gehandelt. Das lag in erster Linie am **Torch Framework** [<https://torch.ch/>] – eine der populärsten Machine-Learning-Bibliotheken sowohl für den produktiven Einsatz als auch für Forschungszwecke. Wenn Sie in ältere DeepLearning-Modelle abtauchen, finden sich oft zahlreiche Verweise auf Torch und Lua-Quellcode. Es könnte durchaus nützlich sein, sich etwas Knowhow über die Torch API anzueignen, die einige Ähnlichkeiten zur Basis-API von PyTorch aufweist. Wenn Sie allerdings kein gesteigertes Bedürfnis haben, für Ihre Applikationen in historische Forschung abzutauchen, können Sie auf Lua auch gut und gerne verzichten.

Julia [<https://julialang.org/>]

Bei Julia handelt es sich um eine High-Performance-Programmiersprache, die ihren Fokus auf numerische Berechnungen legt. Dadurch passt sie auch wunderbar in die mathematisch ausgerichtete Welt der Künstlichen Intelligenz. Julia mag derzeit nicht die populärste Coding-Sprache sein, allerdings bieten Wrapper wie **TensorFlow.jl** [<https://github.com>]

b.com/malmaud/TensorFlow.jl] und [Mocha \[https://github.com/pluskid/Mocha.jl\]](https://github.com/pluskid/Mocha.jl) guten Deep-Learning-Support. Wenn das relativ kleine Ökosystem kein Ausschlusskriterium für Sie darstellt und Sie von Julias Fokus auf High-Performance-Berechnungen profitieren wollen, sollten Sie einen Blick riskieren. (fm)

Dieser Artikel ist im Original [<https://www.infoworld.com/article/2258342/6-best-programming-languages-for-ai-development.html>] bei unserer Schwesterpublikation [Infoworld.com](https://www.infoworld.com) erschienen.

Artificial Intelligence[<https://www.computerwoche.de/artificial-intelligence/>]

Generative AI[<https://www.computerwoche.de/generative-ai/>]

NEWSLETTER

First Look

Von KI bis Business Software: Mit unserem täglich versandten First Look Newsletter bleiben Sie auf dem Laufenden.

Email Adresse



Mit der Übermittlung Ihrer **Daten** erklären Sie sich mit unserer DATENSCHUTZERKLÄRUNG [<https://foundryco.com/privacy-policy/>] einverstanden.

JETZT ANMELDEN

Über Uns



Rechtliches



Mehr



Unser Netzwerk



© 2026 FoundryCo, Inc. All Rights Reserved.

u

